Sprint 6 -
Drupal essentials Report

Prepared by: Aicha Nait Dayaa

Supervised by: Mr. Abdelmajid Bendrif
Mr. Hamza Bahlaouane

Date: February 28, 2026

Contents

1 Introfuction . . . 3
2 Installation guide . . . 3
2.1 Setting up the database . . . 3
2.2 Creating the project directory and necessary dependencies . . . 3
2.3 Configuring Apache Virtualhost . . . 4
2.4 Setting up file Permissions . . . 4
2.5 Creating a Node Using the Basic Page Content Type . . . 5
2.6 Users Configuration . . . 6
2.7 Cron settings . . . 6
2.8 Checking Status Report . . . 7
3 Translation . . . 8
3.1 Enabling Translation Support . . . 8
3.2 Adding a new language . . . 8
3.3 Language detection methods . . . 8
3.4 Language detection and selection . . . 8
3.5 Enabling translation on the Article content type . . . 8
3.6 Testing Translation on a new article . . . 8
4 Media . . . 9
4.1 Enabling the media modules . . . 9
4.2 Uploading a csv file . . . 9
4.3 Uploading to the private files folder . . . 10
4.4 File entity module . . . 11
4.5 Adding a media field to the Article content type . . . 11
5 Step Back . . . 11
5.1 Cookies . . . 11
5.2 reCAPTCHA . . . 12
5.3 Email Registration . . . 13
5.4 Coffee . . . 13
5.5 Google Sign-In . . . 13
5.6 Additional modules . . . 14
6 Emails . . . 14
6.1 Configuring the SMTP transport . . . 15
6.2 Customizing the email template . . . 15
7 SEO . . . 15

7.1 Meta tags 15
7.2 URL aliases 16
7.3 Redirect 16
7.4 Path Redirect Import 16
8 Cron Jobs 17
8.1 Ultimate Cron 17
8.2 Scheduler 17
8.3 Scheduler Test 17
9 Views 18
9.1 Content types 18
9.2 Listings 20
9.3 Image styles 20
9.4 Blocks 21
9.5 Forms 22

1 Introfuction

In this report, we document the steps followed while exploring the Drupal interface and its core functionalities, following the sprint guide.

2 Installation guide

2.1 Setting up the database

We'll set up a mysql database using docker :

.env file

```
MYSQL_ROOT_PASSWORD=root
MYSQL_DATABASE=new_drupal
MYSQL_USER=drupal_test_user
MYSQL_PASSWORD=1234
```

Docker compose file

```
version : '3.8'
services:
  mysql:
    image: mysql:8.0
    container_name: mysql-drupal
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
volumes:
  mysql_data:
```

2.2 Creating the project directory and necessary dependencies

We'll create the project and the Drupal's Command Line Tool Drush :

```
composer create-project drupal/recommended-project drupal_demo_project
composer resuire drush/drush
```

2.3 Configuring Apache Virtualhost

Now we need to tell Apache about our drupal project by adding a virtualhost entry to the configuration file **httpd.conf**.

```
<VirtualHost *:8081>
    ServerName new-drupal.local
    DocumentRoot "/Users/mac/projects/drupal/drupal_demo_project/web"
    <Directory "/Users/mac/projects/drupal/drupal_demo_project/web">
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

And then we need to add the ServerName to **/etc/hosts**. Since `new-drupal.local` is a custom local domain, this file acts as a local DNS override, manually mapping the domain to `127.0.0.1`; our own machine.

```
127.0.0.1   new-drupal.local
```

2.4 Setting up file Permissions

And now we need to set the correct file permissions. We start by creating the **web/sites/default/files** directory, which is where Drupal stores all uploaded and generated files at runtime. Without it, any feature involving file handling will fail.

We then copy **default.settings.php** to **settings.php**.

Finally we set the correct permissions so that Apache has the right level of access to write into both files during and after installation.

```
mkdir -p web/sites/default/files
cp web/sites/default/default.settings.php web/sites/default/settings.
   php
chmod 664 web/sites/default/settings.php
chmod -R 755 web/sites/default/files
```

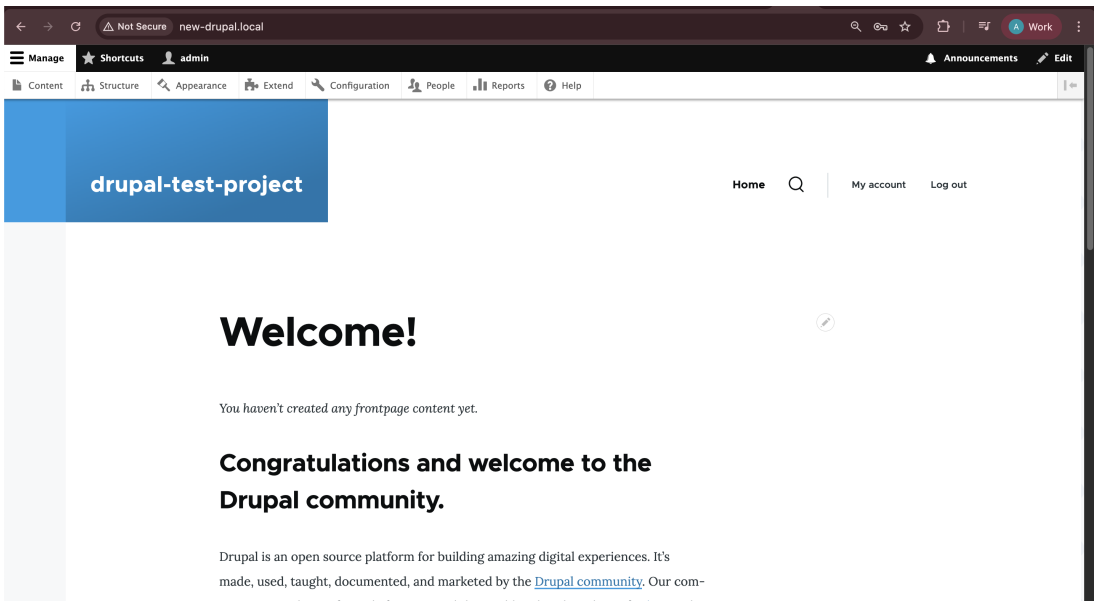


Figure 1: Caption

2.5 Creating a Node Using the Basic Page Content Type

A node in Drupal is any piece of content. The Basic Page content type (page) is built for static content like About pages, Contact pages, etc.

We navigate to `/node/add/page` to create our first node.

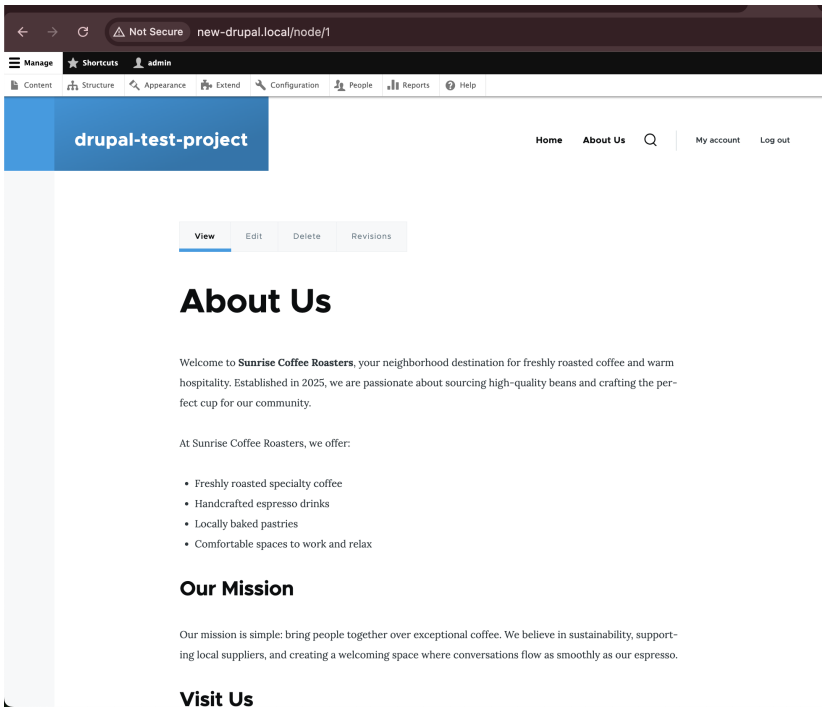


Figure 2: About Us Page

Setting the default front page

Once the node is created, we navigate to `/admin/config/system/site-information` and set the **Default front page** field to `/node/1`, making our newly created page the homepage of the site.

2.6 Users Configuration

Account registration permissions

We configure who is allowed to register an account on `http://localhost/en/admin/config/people/accounts`. We set this to *Visitors*, but require administrator approval before the account becomes active. This prevents unauthorized access while still allowing self-registration.

Creating the Webmaster role and assigning it to a user

We create a new role named **Webmaster** at `http://localhost/en/admin/people/roles/add`, which carries the same permissions as a content editor. We then create a new user and assign them this role.

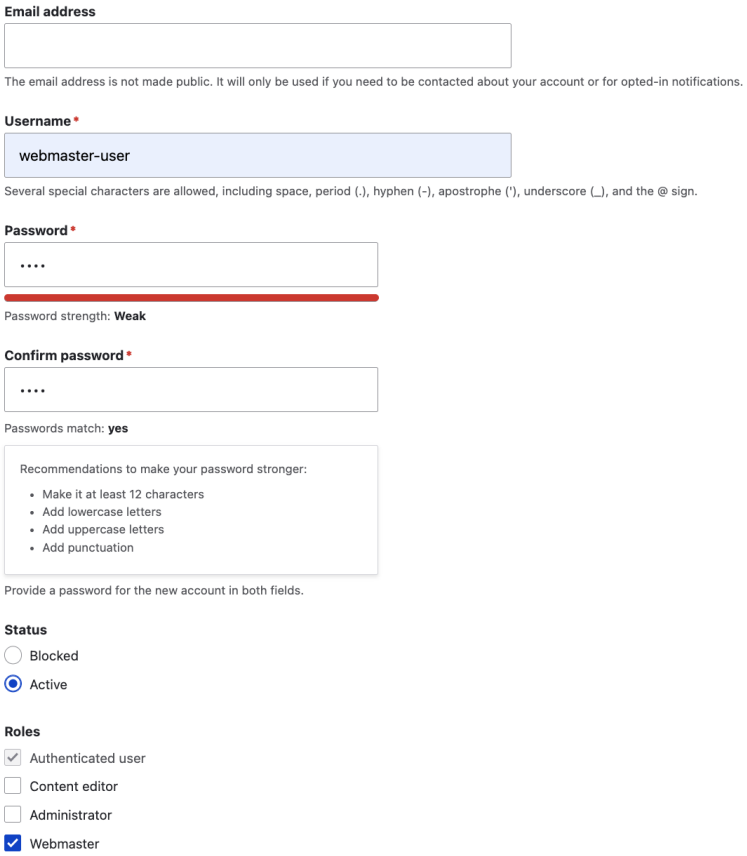


Figure 3: Creating the Webmaster role and assigning it to a new user

2.7 Cron settings

We navigate to `/admin/config/system/cron` and set **Run cron every** to *Never*. The motive behind this is important: on a proper Linux server, cron jobs should be scheduled at the operating system level using `crontab`, rather than being triggered by user page visits.

2.8 Checking Status Report

We navigate to `/admin/reports/status` to review the overall health of the Drupal installation.

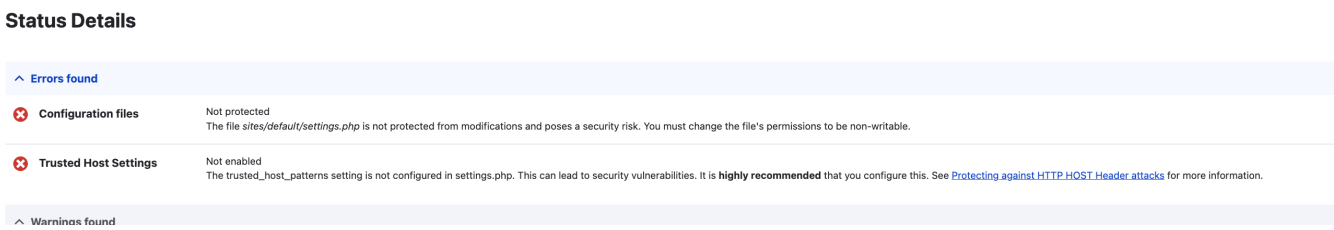


Figure 4: Status Report

We address the two errors reported:

Error 1: settings.php is not protected from modifications

The file `sites/default/settings.php` is writable, which poses a security risk. The fix is to remove write permissions:

```
chmod -w settings.php
```

After applying this change, the file permissions are:

```
-r-xr-xr-x@  1 mac  staff  36727 23 Feb 11:31 settings.php
```

Error 2: trusted_host_patterns is not configured

The `trusted_host_patterns` setting is missing from `settings.php`. This setting protects Drupal against HTTP Host header injection attacks by explicitly whitelisting the domain names the site should respond to. We fix this by adding the appropriate `trusted_host_patterns` configuration directly to `settings.php`:

```
* @endcode
* will allow the site to run off of all variants of example.com and
* example.org, with all subdomains included.
*
* @see https://www.drupal.org/docs/installing-drupal/trusted-host-settings
*/
$settings['trusted_host_patterns'] = [
        '^new-drupal\.local$',
        '^localhost$',
        '^127\.0\.0\.1$',
 ];
```

Figure 5: Adding trusted_host_patterns to settings.php

3 Translation

3.1 Enabling Translation Support

We enable the necessary Drupal translation modules using Drush:

```
./vendor/bin/drush en config_translation content_translation
```

3.2 Adding a new language

We navigate to `/admin/config/regional/language` and add French as a new language, keeping English as the default.

3.3 Language detection methods

We then visit `/admin/config/regional/language/detection` to review the available language detection methods. Drupal supports several methods for determining which language to serve:

- **URL** — detects language from a path prefix or domain (e.g. `/fr/...`)
- **Session** — stores the language preference in the user's session
- **User preferences** — uses the language set in the authenticated user's account
- **Browser** — reads the `Accept-Language` header sent by the browser

We configure the URL detection method and set `en` as the path prefix for English and `fr` for French so that both languages are explicitly prefixed in the URL.

3.4 Language detection and selection

Drupal separates language detection into two independent scopes:

- **Interface text language detection** determines the language used to render the Drupal interface itself — menus, buttons, labels, and any translatable UI strings.
- **Content language detection** determines the language used to serve the actual content (nodes, fields, etc.).

3.5 Enabling translation on the Article content type

We navigate to `/admin/structure/types/manage/article` and enable translation for the Article content type, also activating the language selector on create and edit pages.

We can verify the applied configuration at `/admin/config/regional/content-language`.

3.6 Testing Translation on a new article

We create a new article with the title **English** and add a French translation for it. The article can be accessed at `/en/node/2`, and its French translation at `/fr/node/2`.

However, manually editing the URL path prefix is not a practical way for a regular visitor to switch languages. We therefore install the `lang_dropdown` module to provide a language dropdown switcher in the interface:

```
composer require 'drupal/lang_dropdown:^2.4'
```

And then we do the same for the Basic page content type :

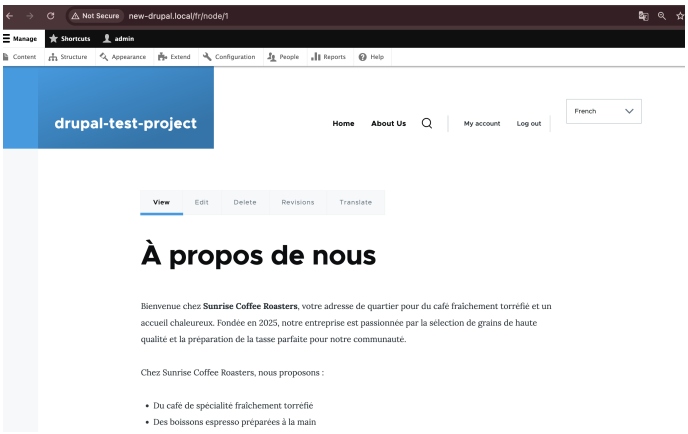


Figure 6: French About us page

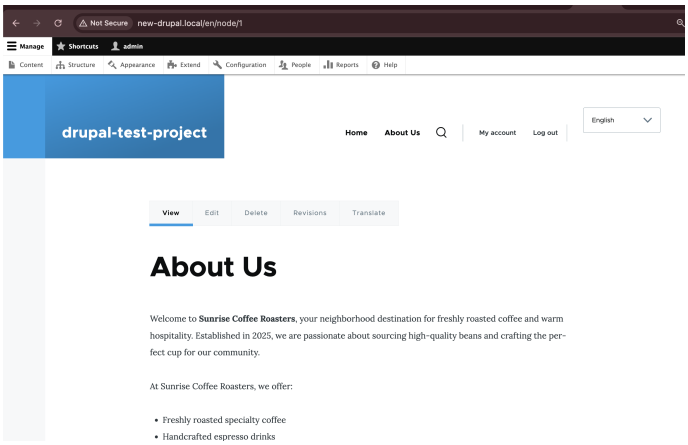


Figure 7: English About us page

4 Media

4.1 Enabling the media modules

We enable the **Media** and **Media Library** modules. Media allows files, images, and documents to be managed as reusable standalone entities, while Media Library adds a browsable UI for selecting and reusing them across content. After enabling both, we update the Webmaster role permissions accordingly.

4.2 Uploading a csv file

When first uploading a csv file, we get an error indicating that the CSV extension is not supported. To solve this issue, we have to add the extension csv to the field field_media_document.

Once uploaded, the file URI follows the pattern `/sites/default/files/(date)/(filename).csv`. This is the default public file storage location, files stored here are served directly via URL, which is convenient for development but not appropriate for sensitive files in production.

Name	MIME type	Size	Status	Upload date	Changed date	Used in	Operations
customers.csv	text/csv	1.63 MB	Permanent	Tue, 24 Feb 2026 - 11:45	Tue, 24 Feb 2026 - 11:46	1 place	Delete

```
~/p/dr/drupal_demo_project/web  ls sites/default/files/2026-02
customers.csv
```

4.3 Uploading to the private files folder

For sensitive files, Drupal supports a private file system where files are not served directly by the web server but are instead routed through Drupal's access control layer.

Setting up the private directory

We create the `sites/default/private` directory and give it the necessary write and execute permissions. To prevent direct web access to this folder, we add a `.htaccess` file that denies all requests:

```
# Turn off all options we don't need.
Options None
Options +FollowSymLinks
# Set the catch-all handler to prevent scripts from being executed.
SetHandler Drupal_Security_Do_Not_Remove_See_SA_2006_006
<Files *>
  SetHandler Drupal_Security_Do_Not_Remove_See_SA_2013_003
</Files>
# Disable PHP engine if possible
<IfModule mod_php.c>
  php_flag engine off
</IfModule>
<IfModule mod_authz_core.c>
  Require all denied
</IfModule>
```

We then register the private path in `settings.php` so that Drupal recognises it:

```
$settings['file_private_path'] = 'sites/default/private';
```

Changing the upload destination

We update the `field_media_document` field settings in `http://localhost/en/admin/config/media/file-s` to use the private file system as the upload destination. After saving, we verify that newly uploaded files are correctly stored under `sites/default/private` and are no longer publicly accessible via a direct URL.

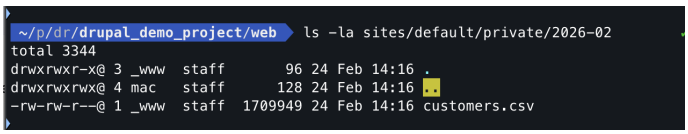


Figure 8: Upload destination set to the private file system

4.4 File entity module

We install the **File Entity** module (`drupal/file_entity`), which extends Drupal's core file handling by turning files into first-class entities (treated the same way as a node). This gives us dedicated file type management at `/admin/structure/file-types`, where each file type (image, video, etc.) can be configured independently, including its associated MIME types visible at `/admin/structure/file-types/manage/document/`.

4.5 Adding a media field to the Article content type

We add a new field named **Photo** of type Media to the Article content type. When comparing this field to the default Image field on the article form at `/node/add/article`, the key difference:

- Unlike the default Image field which requires a new upload each time, the Media field references a Media entity from the Media Library, allowing files to be reused across content.

5 Step Back

5.1 Cookies

Privacy compliance is a legal requirement for most websites today. We install the **EU Cookie Compliance** module, which adds a configurable cookie consent banner to the site.

```
composer require 'drupal/eu_cookie_compliance:^1.29'
./vendor/bin/drush en eu_cookie_compliance
```

After enabling the module, we configure it at `/admin/config/system/eu-cookie-compliance/setting` and place the **EU Cookie Compliance Button Block** in the Footer Bottom region. Testing in a private browser window confirms the cookie consent popup appears correctly.

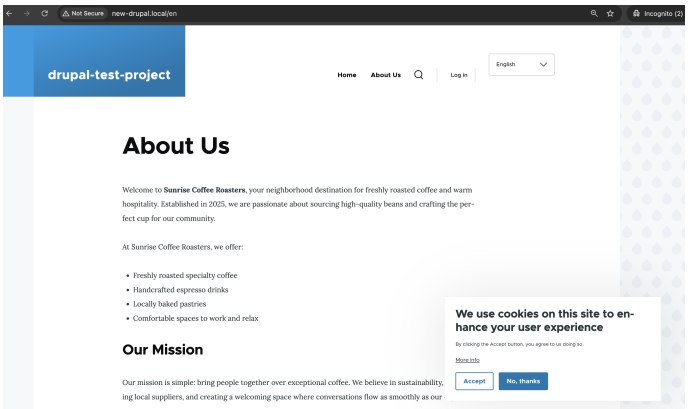


Figure 9: Cookie consent popup

We then translate both the block and its settings to French. When translating the settings at `/admin/config/system/eu-cookie-compliance/settings/translate`, this is **Config Translation** — distinct from the Content Translation we used earlier for nodes. Config Translation covers the translation of module settings and configuration, rather than content entered by users.

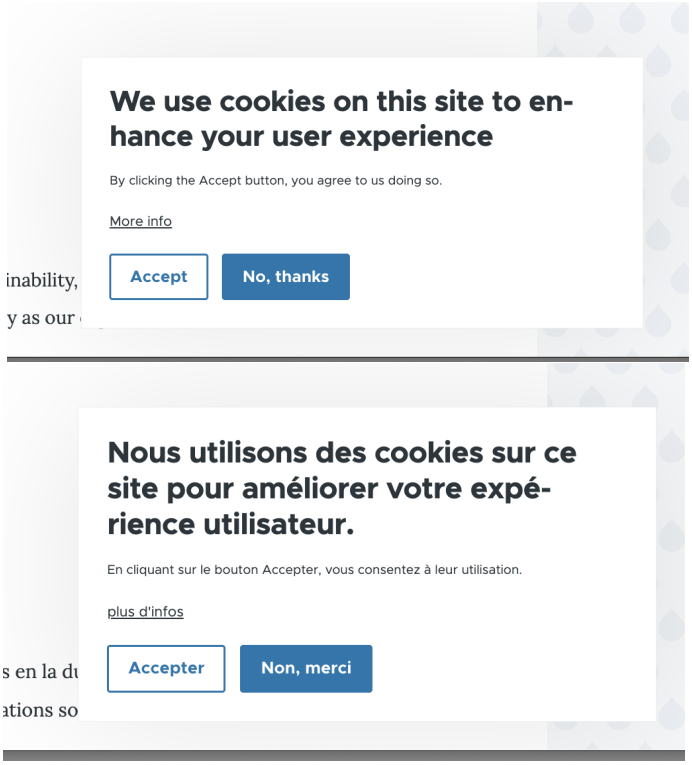


Figure 10: Cookie Translation

5.2 reCAPTCHA

To protect forms from bots, we install the **reCAPTCHA** module (using V2). This requires registering the site on Google's platform to obtain a site key and secret key, specifying the allowed domains (e.g. `localhost`, `127.0.0.1`). These keys are then entered in the module configuration. Finally, we enable reCAPTCHA on the login and registration forms at `/admin/config/people/captcha/capt`

```
composer require 'drupal/recaptcha:^3.4'
./vendor/bin/drush en recaptcha
```

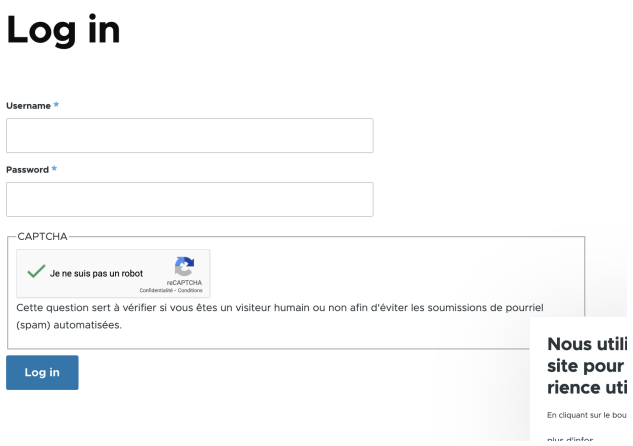


Figure 11: reCAPTCHA enabled on the login form

5.3 Email Registration

By default, Drupal only allows users to log in with their username, which is poor UX. The **Email Registration** module adds the ability to log in using an email address instead.

```
composer require 'drupal/email_registration:^2.0@RC'
./vendor/bin/drush en email_registration
```

5.4 Coffee

Then to facilitate navigation iin the Drupal admin interfac, we'll install the coffee model.

```
composer require 'drupal/coffee:^2.0'
./vendor/bin/drush en coffee
```

5.5 Google Sign-In

To allow users to register and log in via Google, we install the **Social Auth** and **Social Auth Google** modules.

```
composer require 'drupal/social_auth_google:^4.0'
./vendor/bin/drush en social_auth_google
```

We create a Google OAuth client ID and secret in the Google Cloud Console, and register the allowed JavaScript origin and the OAuth callback URL provided by the module, and then enter the client ID and secret at `/admin/config/social-api/social-auth/google`.

We place the **Social Auth Login** block in the Content region, then restrict its visibility to `/user/login` and `/user/register` only, so the Google button does not appear on every page.

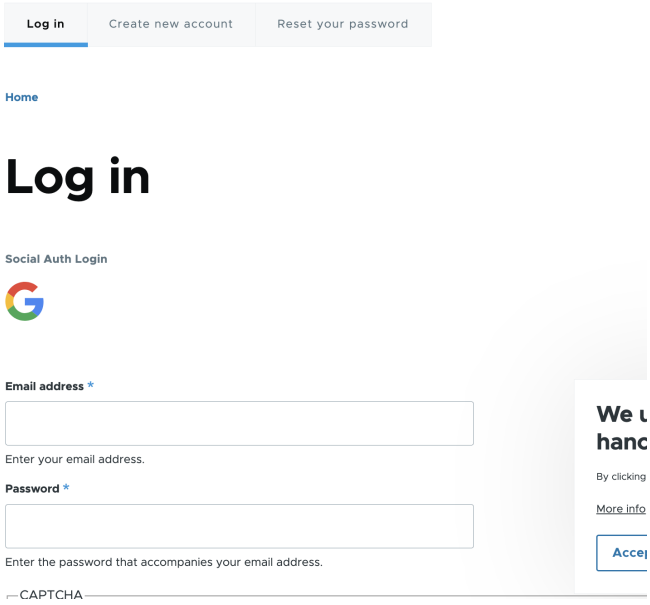


Figure 12: Google Sign-In block on the login page

5.6 Additional modules

Responsive Favicons

Security Review

The **Security Review** module performs an automated audit of the Drupal installation and flags potential security issues.

Security Kit

The **Security Kit** module protects Drupal against common web attacks by adding HTTP security headers and input/output protections.

- We use it in our case, to set the `X-Frame-Options` header, which prevents the site from being embedded in an iframe on another domain, mitigating clickjacking attacks.

6 Emails

By default, Drupal sends emails using PHP's built-in mail function, which delegates to the underlying mail transfer agent configured on the system. There is no way to configure SMTP from the Drupal interface without a module. We install **Symfony Mailer** for this purpose:

```
composer require 'drupal/symfony_mailer:^1.6'
./vendor/bin/drush en symfony_mailer
```

To test email sending locally, we run **MailHog** via Docker :

```
docker run --rm --name mailhog -p 8025:8025 -p 1025:1025 mailhog/
    mailhog
```

6.1 Configuring the SMTP transport

We add a new **SMTP Mailer transport** in the Symfony Mailer settings, pointing to `localhost:1025` with SSL disabled and no credentials, then set it as the default mailer. We verify the setup by sending a test email at `/admin/config/system/mailer/test` and confirming it appears in the MailHog interface.

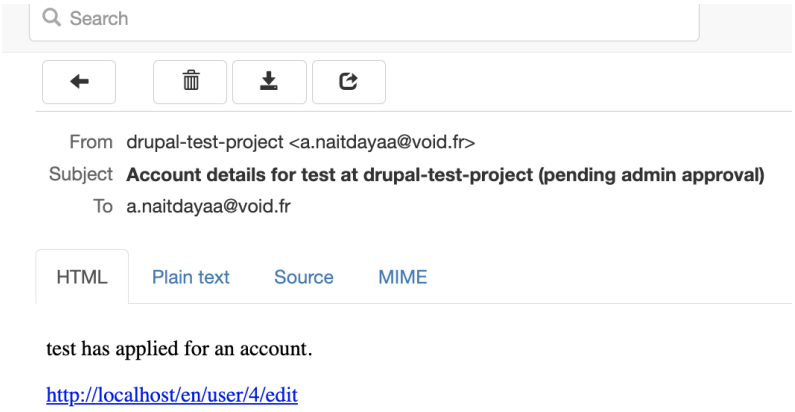


Figure 13: Test email received in MailHog

6.2 Customizing the email template

We navigate to `/admin/config/people/accounts`, override the email formatter for the **Welcome (awaiting approval)** template, and paste in the HTML. We then trigger a new registration to confirm the email is now rendered in HTML.

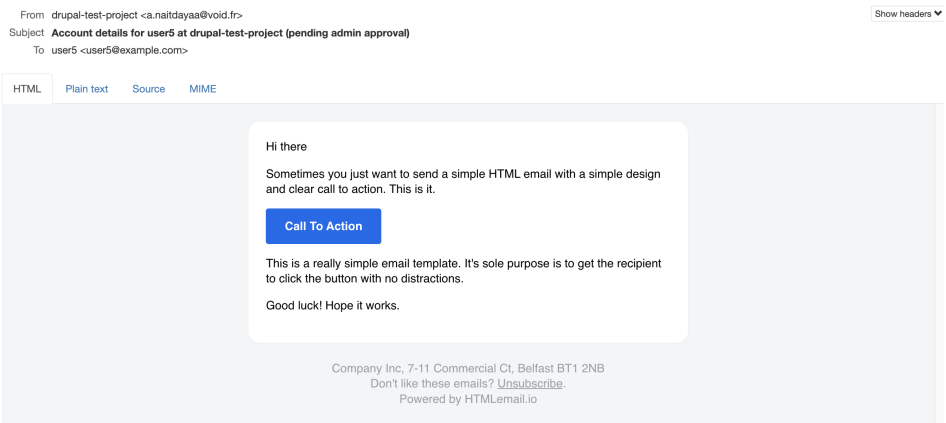


Figure 14: HTML email template applied to the Welcome email

7 SEO

7.1 Meta tags

By default, Drupal's HTML output contains no Open Graph tags and no way to control the description meta tag. The **Metatag** module adds this capability, and also uses tokens to dynamically populate meta tag values.

```
composer require 'drupal/metatag:^2.1'
./vendor/bin/drush en metatag metatag_open_graph -y
```

We then add a **Meta tags** field to the Basic Page content type at `/admin/structure/types/manage/page/fields/add-field`, making sure to enable the option *Users may translate this field* so meta tags can be translated per language. The field then appears in the node edit form sidebar when creating or editing a page.

7.2 URL aliases

By default, nodes are accessible at paths like `/node/1`, `/node/2`, which are meaningless to both users and search engines. We can manually add aliases, but this is tedious and error-prone. The **Pathauto** module automates this by generating aliases from the node title using token-based patterns.

```
composer require 'drupal/pathauto:^1.14'
./vendor/bin/drush en pathauto
```

We configure two patterns at `/admin/config/search/path`:

- Basic page: `/[node:title]`
- Article: `/articles/[node:title]`

Using different patterns per content type allows us to distinguish between them by URL structure. After creating a test article, the alias is automatically generated as `/en/articles/article-aliases-test`.

7.3 Redirect

At this point, both `/node/6` and `/articles/article-aliases-test` point to the same content. Search engines don't like duplicate URLs, and the canonical URL should be the only one accessible. The **Redirect** module handles this by automatically redirecting node paths to their alias equivalent.

```
composer require 'drupal/redirect:^1.12'
./vendor/bin/drush en redirect
```

The option responsible for this behaviour is *Enforce clean and canonical URLs* at `/admin/config/search/redirect/settings`. Once enabled, visiting `/node/6` automatically redirects to `/articles/article-aliases-test`.

7.4 Path Redirect Import

For large migrations involving hundreds or thousands of URLs, adding redirects one by one is impractical. The **Path Redirect Import** module allows bulk importing redirect rules from a CSV file containing source and destination paths.

```
composer require 'drupal/path_redirect_import:^2.1'
./vendor/bin/drush en path_redirect_import -y
```

8 Cron Jobs

Previously, the Drupal internal cron was disabled intentionally. When Drupal cron is triggered by a user's HTTP request, the page must wait for all cron tasks to finish before being served. This creates a poor user experience, especially when cron includes long-running background tasks.

The solution is to fully decouple cron from HTTP requests by disabling it in Drupal and scheduling it at the operating system level using `crontab`.

8.1 Ultimate Cron

To get an overview of all registered cron jobs on the system, we install the **Ultimate Cron** module, which provides a listing at `/admin/config/system/cron/jobs`.

```
composer require 'drupal/ultimate_cron:^2.0@beta'
./vendor/bin/drush en ultimate_cron -y
```

8.2 Scheduler

We want to have the ability to schedule content for automatic publishing or unpublishing at a future date and time, the **Scheduler** module provides this functionality.

```
composer require 'drupal/scheduler:^2.2'
./vendor/bin/drush en scheduler
```

We enable scheduled publishing and unpublishing on the Article content type at `/admin/structure/types/manage/article`.

8.3 Scheduler Test

We create a new article and set a *Publish on* date 5 minutes into the future. As an admin, the article is visible immediately. Visiting the same URL as an anonymous user in a private browser returns an Access Denied response, confirming the article is not yet publicly accessible.

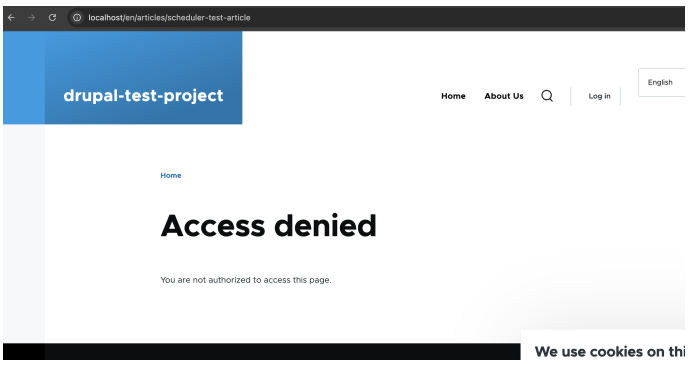


Figure 15: Access Denied on an unpublished article

Since the Drupal internal cron is disabled, the scheduled publication does not fire automatically. We trigger it manually using the Drush command:

```
./vendor/bin/drush scheduler:cron
[notice] Lightweight cron run activated by drush command.
[notice] Article: scheduled publishing of Scheduler Test article.
[notice] Lightweight cron run completed.
[notice] Message: Scheduler lightweight cron completed
```

Revisiting the article as an anonymous user now returns the published content as expected.

9 Views

9.1 Content types

Vocabularies

We create two vocabularies at `/admin/structure/taxonomy/add`: **Categories** and **Thématiques**, both with translation enabled.

News content type

We create a new content type **News** at `/admin/structure/types/add` with the following fields:

- **Body** — Formatted text (long, with summary), Basic HTML, required, translatable
- **Image** — Media field (machine name: `field_news_image`), required, media type: Image
- **Category** — Taxonomy term reference, required, vocabulary: Categories, widget: Select List
- **Thématique** — Taxonomy term reference, required, vocabulary: Thématiques, widget: Autocomplete
- **Meta Tags** — for SEO control per node

We group the Category and Thématique fields using the **Field Group** module, which allows fields to be visually grouped in the form display:

```
composer require 'drupal/field_group:^4.0'
./vendor/bin/drush en field_group -y
```

We also configure a Pathauto pattern for the News content type: `/news/[node:title]`.

Generating dummy content

Rather than manually creating content, we use the **Devel** module and its **devel_generate** submodule, which can bulk generate nodes, taxonomy terms, media, and more with dummy data.

```
composer require 'drupal/devel:^5.5'
./vendor/bin/drush en devel devel_generate -y
```

We navigate to `/admin/config/development/generate/` and generate:

- 20 terms for each vocabulary (max 6 characters per term name, flat list, no hierarchy)
- 20 media items
- 9 News nodes

Drupal entity system

This is a good moment to step back and look at what we have built through the lens of Drupal's entity system. Drupal organises all data as entities, split into two categories:

Concept	Drupal Entity Type	Description
Configuration Entities	ConfigEntityType	Define structure & settings for the content
Vocabulary	ConfigEntityType (taxonomy_vocabulary)	Defines a taxonomy vocabulary used to classify content.
Content Type	ConfigEntityType (node_type)	Defines a structure for content such as Article or Basic Page.
Fields	ConfigEntityType (field_storage_config, field_config)	Define reusable fields attached to entities.
Content Entities	ContentEntityType	Hold actual user generated content.
Node	ContentEntityType (node)	Actual content entry created by users (e.g., an article).
Term	ContentEntityType (taxonomy_term)	Actual terms within a vocabulary, like "News" or "Sports"

Table 1: Drupal Entity System Overview

Also, it is worth mentioning the distinction between **Manage form display** and **Manage display**. The former controls how fields appear when creating or editing content in the back-office, and the latter controls how fields are rendered on the frontend.

9.2 Listings

In this section, we discovered how to create views, and applying configuration, in order to display a news view, that bundles many pages.

Creating the News view

We create a new view at `/admin/structure/views/add` named **News**, showing Content of type News, with a page display (Responsive Grid of Teasers, 3 columns, 9 items per page, full pager) at path `/news`, and a block display.

We configure the view with exposed filters on Category, Thématique, and Title, an exposed sort on date of creation, and a no results message.

Fixing the layout

Visiting `/en/news` reveals a broken grid layout and overly long body text. We install **Asset Injector** to inject page-specific CSS, and **Smart Trim** to truncate the body field output:

```
composer require drupal/asset_injector drupal/smart_trim
./vendor/bin/drush en asset_injector smart_trim -y
```

We add the following CSS rule via Asset Injector, restricted to the `/news` page:

```
.view, .view-content {
    display: block !important;
}
```

In the Teaser display mode of the News content type, we configure the body field to use the *Smart Trimmed* formatter with a 20 character limit, hide all field labels, and set the field order to: Image, Category, Body, with all remaining fields disabled.

9.3 Image styles

The generated images all have different dimensions, which breaks the view of the grid. Instead of manually resizing images, Drupal can automatically generate resized versions using Image Styles. We create a new image style at `/admin/config/media/image-styles` named **News Teaser Image**, using a Scale & Crop effect at 294×165px anchored to the center, and apply it to the Image field.

Responsive images

Each image is currently served at the same size regardless of the user's screen. To deliver appropriately sized images per viewport, we enable the **Responsive Image** module.

```
./vendor/bin/drush en responsive_image -y
```

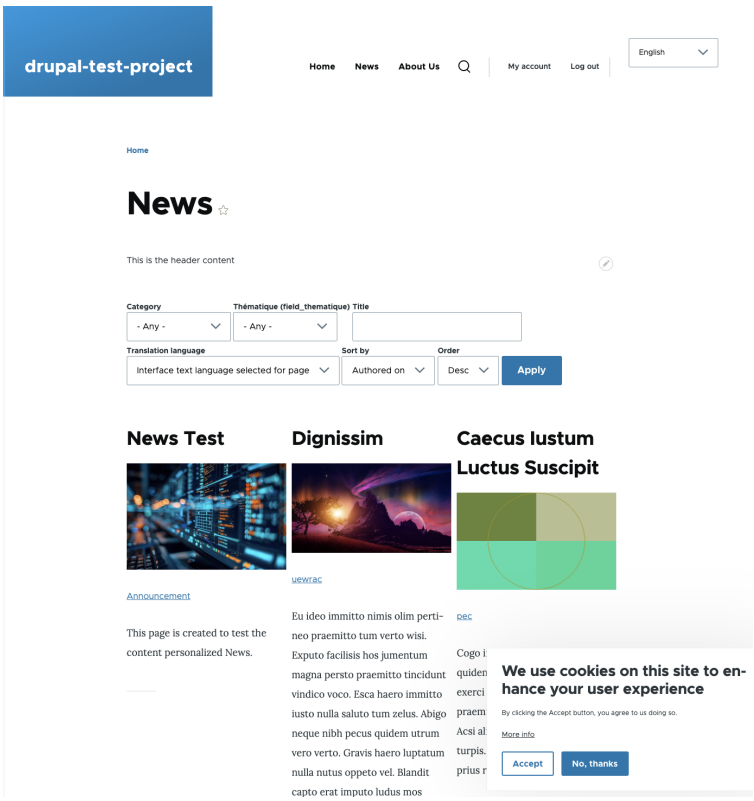
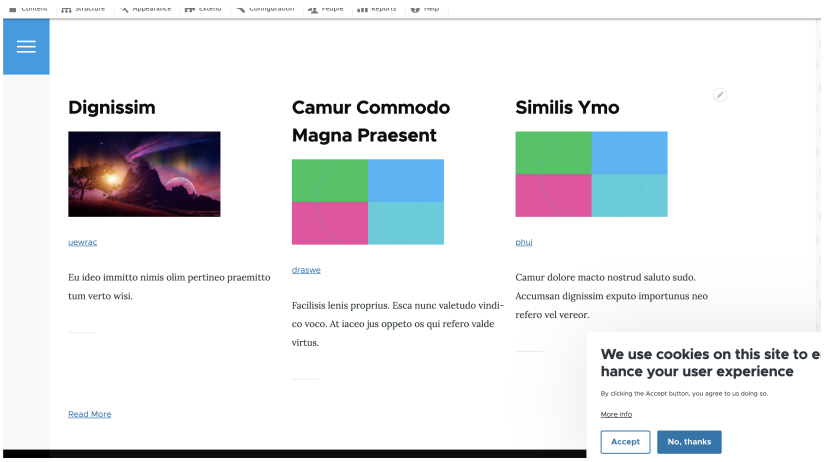


Figure 16: News View

9.4 Blocks

- Created a Views block with the following configuration:
  - Format: Responsive grid
  - Display mode: Content type teaser
  - Addeded a “More” link
- Placed the block in Block Layout:
  - Region: Content Below
  - Restricted content to News content type
  - Set items per block to 3
- Customized the order of items in the block:
  - Created an Entity Queue
  - Added news items to the queue
  - Enabled auto-add for new content
  - Added a relationship in Views from the block to the entity queue to use the queue order



9.5 Forms

One of the most used modules in form creation is Webform, which provides a user interface to build complex forms without writing large amounts of code.

To start, we install the Webform module using Composer and enable it using Drush.

```
composer require 'drupal/webform:^6.3@beta'
./vendor/bin/drush en webform webform_ui
```

After installation, we can access the Webform administration interface at: `/en/admin/structure/webform`.

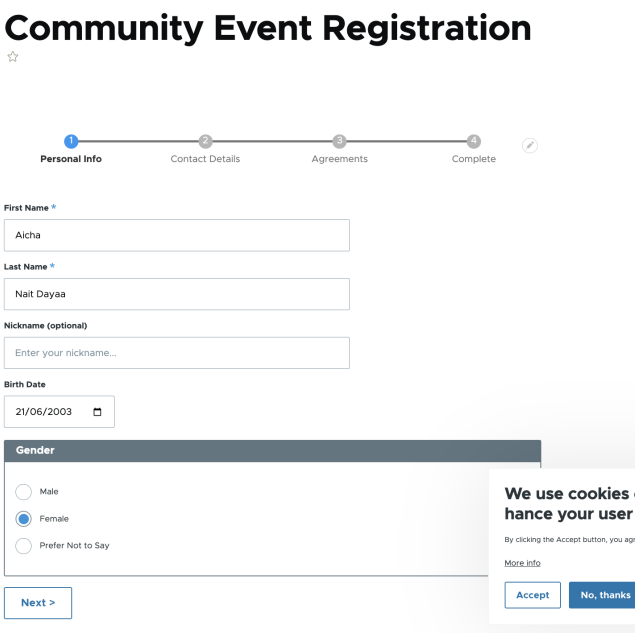
From there, we can create a new form and define its structure. In our example, the form is divided into several steps to create a wizard multi-step form. The pages used in this example are: **Personal Information**, **Contact Details**, and **Agreements**.

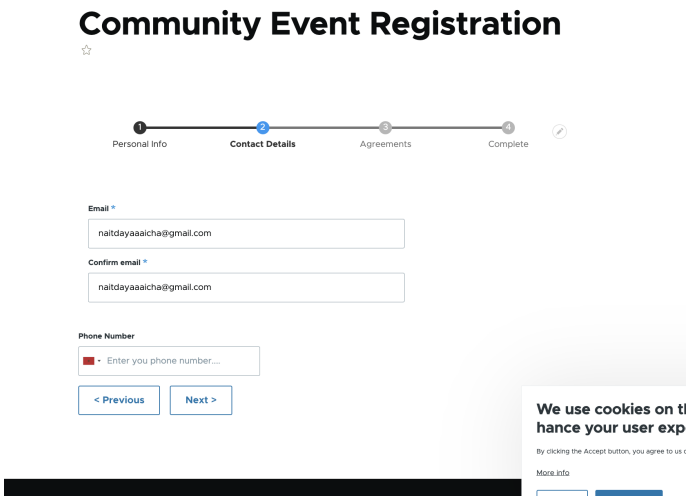
Each page contains several form elements such as **Text fields, Textareas, Radio buttons, Email fields, etc.**. These elements allow users to input and submit the required information in an organized way.

For telephone number validation, we installed the following module:

```
composer require 'drupal/telephone_validation:^2.5'
./vendor/bin/drush en telephone_validation
```

This module ensures that telephone numbers follow a valid format before the form can be submitted.


Community Event Registration
Personal Info
Contact Details
Agreements
Complete
First Name *
Aicha
Last Name *
Nait Dayaa
Nickname (optional)
Enter your nickname...
Birth Date
21/06/2003
Gender
Male
Female
Prefer Not to Say
Next >
We use cookies
hance your user
By clicking the Accept button, you agr
More info
Accept
No, thanks


Community Event Registration
Personal Info
Contact Details
Agreements
Complete
Email *
naitdayaaaicha@gmail.com
Confirm email *
naitdayaaaicha@gmail.com
Phone Number
Enter you phone number....
< Previous
Next >
We use cookies on tl
hance your user exp
By clicking the Accept button, you agree to us
More info

Community Event Registration
Personal Info
Contact Details
Agreements
Complete
I agree to the event's rules *
< Previous
Submit Registration